



Capstone Project Architecture

End-to-End System Design + Real Case Studies

Java Backend Focus



Session Structure

1. Capstone project overview
2. Architecture blueprint
3. Database & scaling decisions
4. Concurrency & performance design
5. Observability & reliability
6. Case studies discussion
7. Trade-offs & real-world lessons



Capstone Objective

Build a scalable backend system using:

- Java 8+
- Spring Boot
- REST APIs
- Microservices architecture
- Database + caching
- Concurrency principles
- Deployment-ready design



Capstone Example Project

E-Commerce Backend System

Core services:

- User Service
- Product Service
- Order Service
- Payment Service
- Inventory Service
- Notification Service



High-Level Architecture

Client

- API Gateway
- Microservices
- Databases
- Cache
- Message Queue



API Gateway Responsibilities

- Authentication
- Routing
- Rate limiting
- Request aggregation
- Logging



Service Layer Architecture

Each service contains:

Controller → Service → Repository → Database

Clear separation of concerns.



Database Design (Example: Order Service)

Table: Orders

id | user_id | total_amount | status | created_at

Indexes:

- user_id
- created_at



Data Consistency Strategy

Use:

Event-driven architecture.

Order placed → Publish event → Inventory updated → Payment processed.



Event Communication

Use Kafka / RabbitMQ.

OrderCreatedEvent → InventoryService consumes.

Loose coupling.



Caching Strategy

Use Redis for:

- Frequently accessed products
- Session storage
- Rate limiting counters



Scaling Strategy

Horizontal scaling:

- Multiple service instances
- Load balancer
- Auto-scaling via Kubernetes



Concurrency Design

Use:

- `ExecutorService` for async tasks
- Thread-safe collections
- Optimistic locking for DB updates



Performance Optimization

- Use connection pooling
- Database indexing
- Pagination for large datasets
- Avoid N+1 queries
- Use async processing



Reliability Patterns

- Circuit breaker
- Retry mechanism
- Timeout handling
- Health checks
- Graceful shutdown



Observability

Logging:

Centralized logging (ELK)

Monitoring:

Prometheus + Grafana

Metrics:

- Response time
- Error rate
- CPU/memory usage



Security Design

- JWT authentication
- Role-based access
- Input validation
- HTTPS
- API throttling



Deployment Architecture

Docker containers

Kubernetes cluster

Environments:

- Dev
- Staging
- Production



Case Study 1

High Traffic Sale Event

Problem:

10x traffic spike.

Solution:

- Auto-scale services
- Cache popular products
- Use CDN
- Queue-based order processing

Case Study 2

Payment Service Failure

Problem:

External payment gateway down.

Solution:

- Circuit breaker
- Retry with exponential backoff
- Eventual consistency
- Alerting system



Case Study 3

Database Bottleneck

Problem:

Slow queries.

Solution:

- Add index
- Analyze execution plan
- Introduce read replicas
- Use caching layer



Trade-Off Analysis

Consistency vs Availability

Latency vs Throughput

Memory vs Speed

Simplicity vs Scalability



Interview-Level Discussion

Explain:

- Why microservices?
- Why not monolith?
- How to scale?
- How to prevent cascading failures?
- How to monitor production?



Capstone Code Structure Example

```
com.project
├─ controller
├─ service
├─ repository
├─ entity
├─ config
└─ util
```




Capstone Checklist

- Clean architecture
- Proper layering
- Thread safety
- Efficient queries
- Logging
- Error handling
- Unit tests
- Integration tests



Performance & JVM Considerations

- Proper heap sizing
- GC tuning (G1)
- Avoid memory leaks
- Monitor thread pools
- Profile regularly



Real-World Lessons

- Simplicity scales better
- Measure before optimizing
- Failures will happen
- Monitoring is mandatory
- Design evolves over time



Final Takeaways

Capstone integrates:

- DSA knowledge
- Concurrency
- Design patterns
- System design
- Microservices
- JVM tuning
- Real production thinking



Final Takeaways

You are now capable of:

- Designing scalable backend systems
- Explaining trade-offs
- Handling interview system design rounds
- Architecting production-ready Java systems